Cybersecurity Portfolio: Apply Filters to SQL Queries

Role: Information Security Analyst

Database: Organization Relational Database (MariaDB / MySQL)

Target Tables: log_in_attempts, employees

1. Project Description

As a security analyst at a large organization, my role involves actively monitoring security events, investigating suspicious user activity, and ensuring all organizational assets adhere to current security standards. Recently, our security operations center flagged anomalous authentication activities and required updates to employee devices. The goal of this project was to leverage SQL queries with targeted filtering logic (using AND, OR, NOT, and the LIKE operator) on the log_in_attempts and employees tables to extract relevant incident data and identify endpoints requiring mitigation.

2. Retrieve After-Hours Failed Login Attempts

To investigate a potential security incident that occurred outside regular operating hours, I needed to isolate all failed login attempts occurring after 18:00.

```
SELECT *
FROM log_in_attempts
WHERE login_time > '18:00' AND success = 0;
```

Explanation: The WHERE clause combines two conditions with the AND operator. The condition login_time > '18:00' filters for attempts occurring after 6:00 PM, while success = 0 isolates unsuccessful authentication events. Both conditions must evaluate to true for a record to be returned.

3. Retrieve Login Attempts on Specific Dates

A suspicious incident occurred on May 9, 2022. To investigate associated activity, I inspected all authentication events that took place on that date or the preceding day (May 8, 2022).

```
SELECT *
FROM log_in_attempts
WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

Explanation: The OR operator allows records matching either the target date ('2022-05-09') or the day prior ('2022-05-08') to be retrieved, ensuring full visibility into activity spanning the entire incident window.

4. Retrieve Login Attempts Outside of Mexico

Investigation confirmed that the suspicious activity did not originate from within Mexico. Therefore, authentication traffic originating outside Mexico needed to be analyzed. Country values in the table included abbreviations like MEX as well as full names like MEXICO.

```
SELECT *
FROM log_in_attempts
WHERE NOT country LIKE 'MEX%';
```

Explanation: The LIKE operator paired with the wildcard symbol % matches any string beginning with "MEX" (encompassing both MEX and MEXICO). Prepending the NOT operator negates this pattern, returning exclusively records originating from all other countries.

5. Retrieve Employees in Marketing

The security team scheduled endpoint updates for devices belonging to Marketing staff located in the East building. I queried the employees table to obtain their account and office details.

```
SELECT *
FROM employees
WHERE department = 'Marketing' AND office LIKE 'East%';
```

Explanation: The query applies department = 'Marketing' to target the specific organizational unit and joins it via AND with office LIKE 'East%'. The wildcard character matches all room and wing formats prefixed with "East" (such as East-170 or East-320).

6. Retrieve Employees in Finance or Sales

A separate security patch needed to be deployed to all computers assigned to staff in either the Sales or Finance departments.

```
SELECT *
FROM employees
WHERE department = 'Sales' OR department = 'Finance';
```

Explanation: The OR operator evaluates records so that an employee belonging to either the Sales team or the Finance team is returned in the final result set.

7. Retrieve All Employees Except Information Technology

A company-wide configuration update was already applied to the Information Technology department, meaning devices across all other departments required the update.

```
SELECT *
FROM employees
WHERE NOT department = 'Information Technology';
```

Explanation: The NOT operator in conjunction with department = 'Information Technology' filters out all IT personnel, providing a list of all remaining staff across the enterprise.

8. Summary

In this project, I used SQL queries to extract security-critical insights from relational database tables. By applying Boolean logic operators (AND, OR, NOT) and pattern matching (LIKE with %), I investigated after-hours authentication anomalies, reviewed incident timeframes, filtered external geographic traffic, and identified target employee groups for endpoint security updates. These analytical queries demonstrate how database filtering directly supports incident response and enterprise vulnerability mitigation.